

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.*
(BL2, BL3)

Activity Tool : Experiments (High)
Assessment Tool Weightage: 40%

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.	BL2 – Understand	5
2	Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.	BL3 – Apply	5
3	Given relation R(A,B,C,D,E) with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies.	BL3 – Apply	5
4	Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.	BL3 – Apply	5
5	Verify whether R(A,B,C,D) with FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ is in BCNF. If not, decompose it into BCNF.	BL3 – Apply	5
6	Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.	BL3 – Apply	5

7	Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.	BL3 – Apply	5
8	Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).	BL3 – Apply	5
9	Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.	BL2 – Understand	5
10	Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.	BL3 – Apply	5

Code of Conduct:

I R. Indhu (192524332) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

R. Indhu

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results

Q1. STUDENT_COURSE — Functional Dependencies & Candidate Keys

Given Relation

STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade)

The relation stores information about students, courses, instructors, and the grades obtained by students in particular courses.

Given Functional Dependencies

1. **SID $\rightarrow$ SName**
2. **CID $\rightarrow$ CName**
3. **CID $\rightarrow$ Instructor**
4. **SID, CID $\rightarrow$ Grade**

These dependencies mean:

- A particular **SID** uniquely identifies the student's name.
- A particular **CID** uniquely identifies the course name.
- A particular **CID** also determines the instructor teaching that course.
- A combination of **SID and CID** determines the grade obtained by that student in that course.

Finding the Candidate Key

We calculate the closure of {SID, CID}.

Starting with:

(SID, CID)⁺ = {SID, CID}

Using:

SID $\rightarrow$ SName

we get:

(SID, CID)⁺ = {SID, CID, SName}

Using:

CID $\rightarrow$ CName

we get:

(SID, CID)⁺ = {SID, CID, SName, CName}

Using:

CID $\rightarrow$ Instructor

we get:

(SID, CID)+ = {SID, CID, SName, CName, Instructor}

Using:

SID, CID → Grade

we get:

(SID, CID)+ = {SID, CID, SName, CName, Instructor, Grade}

Therefore, {SID, CID} determines every attribute in the relation.

Why SID Alone Is Not a Candidate Key?

SID+ = {SID, SName}

It cannot determine CID, CName, Instructor, or Grade.

Therefore, SID alone is not a candidate key.

Why CID Alone Is Not a Candidate Key?

CID+ = {CID, CName, Instructor}

It cannot determine SID, SName, or Grade.

Therefore, CID alone is not a candidate key.

Final Answer

Candidate Key = {SID, CID}

The combination of Student ID and Course ID uniquely identifies a student's enrollment and grade in a particular course.

Conclusion: The candidate key of the relation is **(SID, CID)**.

Q2. Convert UNF to 1NF

Given Unnormalized Relation

Consider the following relation:

STUDENT(SID, SName, Courses)

SID	SName	Courses
101	Ravi	DBMS, Java
102	Priya	Python, OS

The Courses attribute contains more than one value in a single cell.

For example, for SID 101, the Courses column contains:

DBMS, Java

This violates the **First Normal Form (1NF)** rule.

Rule of 1NF

A relation is in **1NF** if:

1. Every attribute contains only atomic values.
2. There are no repeating groups.
3. Each cell contains a single value.

Conversion to 1NF

We separate the multiple course values into individual rows.

SID	SName	Course
101	Ravi	DBMS
101	Ravi	Java
102	Priya	Python
102	Priya	OS

Now every cell contains exactly one value.

For example:

- Ravi $\rightarrow$ DBMS
- Ravi $\rightarrow$ Java
- Priya $\rightarrow$ Python
- Priya $\rightarrow$ OS

There are no multiple values inside a single cell.

Why This Is 1NF?

The relation now satisfies atomicity because every attribute contains a single value.

The original:

101 | Ravi | DBMS, Java

has been converted into:

101 | Ravi | DBMS

101 | Ravi | Java

Final Answer

The converted relation is:

STUDENT(SID, SName, Course)

and it is in **1NF** because all attribute values are atomic.

Conclusion: The UNF relation is converted to 1NF by removing repeating or multi-

valued attributes and storing each value separately.

Q3. R(A,B,C,D,E) — Convert to 2NF

Given Relation

R(A, B, C, D, E)

Given Functional Dependencies

- $A \rightarrow B, C$
- $BC \rightarrow D$
- $D \rightarrow E$

We first need to identify the candidate key.

Step 1: Find A+

Start with:

$A^+ = \{A\}$

Using:

$A \rightarrow B, C$

we obtain:

$A^+ = \{A, B, C\}$

Now we have B and C.

Using:

$BC \rightarrow D$

we obtain:

$A^+ = \{A, B, C, D\}$

Using:

$D \rightarrow E$

we obtain:

$A^+ = \{A, B, C, D, E\}$

Therefore:

$A^+ = \{A, B, C, D, E\}$

Since A determines every attribute in the relation, **A is a candidate key.**

Step 2: Check for Partial Dependency

A relation is in **2NF** if:

1. It is already in 1NF.
2. There is no partial dependency of a non-prime attribute on part of a composite candidate key.

Here, the candidate key is:

A

The candidate key contains only **one attribute**.

Therefore, it is impossible to have a partial dependency because there is no proper subset of A.

Therefore

There is **no partial dependency**.

Hence, the relation is already in **2NF**, assuming the given FDs are complete.

Transitive Dependency

Although the relation is in 2NF, there is a transitive dependency:

$A \rightarrow B, C$

$BC \rightarrow D$

$D \rightarrow E$

Therefore:

$A \rightarrow D \rightarrow E$

This means A indirectly determines E through D.

This issue is relevant when converting the relation to **3NF**.

Final Answer

Candidate Key = A

Since A is a single-attribute candidate key, partial dependency is not possible.

Therefore, R is already in 2NF.

However, the relation contains a transitive dependency, so further decomposition may be required to achieve 3NF.

Q4. EMPLOYEE — 3NF Decomposition

Given Relation

EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID)

Functional Dependencies

1. **EmpID $\rightarrow$ EmpName, DeptID**
2. **DeptID $\rightarrow$ DeptName, ManagerID**

Step 1: Identify the Candidate Key

EmpID uniquely identifies an employee.

Therefore:

EmpID $\rightarrow$ EmpName, DeptID

Since DeptID determines department information:

DeptID $\rightarrow$ DeptName, ManagerID

Therefore:

EmpID $\rightarrow$ DeptID $\rightarrow$ DeptName, ManagerID

EmpID determines all attributes.

Hence:

Candidate Key = EmpID

Step 2: Identify Transitive Dependency

We have:

EmpID $\rightarrow$ DeptID

and:

DeptID $\rightarrow$ DeptName, ManagerID

Therefore:

EmpID $\rightarrow$ DeptName, ManagerID

This is a **transitive dependency**.

The department information depends on DeptID rather than directly on EmpID.

This violates the requirements of 3NF because non-key attributes are dependent on another non-key attribute.

Step 3: Decompose the Relation

We divide the original relation into two relations.

Employee Relation

EMPLOYEE(EmpID, EmpName, DeptID)

Here:

EmpID $\rightarrow$ EmpName, DeptID

Candidate key:

EmpID

Department Relation

DEPARTMENT(DeptID, DeptName, ManagerID)

Here:

DeptID $\rightarrow$ DeptName, ManagerID

Candidate key:

DeptID

Final Structure

EMPLOYEE

EmpID EmpName DeptID

DEPARTMENT

DeptID DeptName ManagerID

DeptID in EMPLOYEE acts as a **foreign key** referencing DEPARTMENT.

Conclusion

The transitive dependency is removed by separating department details into a separate relation.

Therefore, the decomposition produces relations that satisfy **3NF**.

Q5. BCNF Verification

Given Relation

R(A, B, C, D)

Functional Dependencies

1. **AB $\rightarrow$ C**

2. **C $\rightarrow$ D**

3. **D $\rightarrow$ A**

BCNF stands for **Boyce-Codd Normal Form**.

BCNF Rule

For every non-trivial functional dependency:

X $\rightarrow$ Y

X must be a **superkey**.

Step 1: Find Candidate Keys

AB+

Start with:

AB+ = {A,B}

Using:

AB → C

we get:

AB+ = {A,B,C}

Using:

C → D

we get:

AB+ = {A,B,C,D}

Therefore:

AB is a candidate key.

BC+

Start with:

BC+ = {B,C}

Using:

C → D

we get:

BC+ = {B,C,D}

Using:

D → A

we get:

BC+ = {A,B,C,D}

Therefore:

BC is also a candidate key.

Step 2: Check BCNF

FD 1: AB → C

AB is a candidate key and therefore a superkey.

So:

AB → C ✓

FD 2: $C \rightarrow D$

C alone does not determine all attributes.

Therefore C is **not a superkey**.

So:

$$C \rightarrow D \text{ X}$$

This violates BCNF.

FD 3: $D \rightarrow A$

D alone does not determine all attributes.

Therefore D is **not a superkey**.

So:

$$D \rightarrow A \text{ X}$$

This also violates BCNF.

Therefore

The original relation is:

NOT IN BCNF

Decomposition Using $C \rightarrow D$

Create:

R1(C,D)

and:

R2(A,B,C)

In R1:

$$C \rightarrow D$$

C is the key of R1.

In R2, the dependency involving AB remains.

Conclusion

The original relation is not in BCNF because C and D are not superkeys for their respective dependencies.

The decomposition helps remove the BCNF violations.

Q6. Lossless-Join Decomposition — Chase Algorithm

Given Relation

R(A,B,C)

Functional Dependency

$A \rightarrow B$

Decomposition

The relation is decomposed into:

- **R1(A,B)**
- **R2(A,C)**

We need to determine whether this decomposition is **lossless**.

Step 1: Create the Chase Table

Create one row for each decomposed relation.

Relation	A	B	C
R1	a	b	c ₂
R2	a	b ₂	c

Here:

- a, b, c represent distinguished symbols.
- Subscripted symbols represent different symbols.

Step 2: Apply Functional Dependency

The functional dependency is:

$A \rightarrow B$

Both rows have the same value for A:

A = a

Therefore, according to the FD $A \rightarrow B$, the B values must be the same.

Initially:

Relation	A	B	C
R1	a	b	c ₂
R2	a	b ₂	c

After applying $A \rightarrow B$, the B values are made equal:

Relation	A	B	C
R1	a	b	c ₂
R2	a	b	c

Step 3: Check the Chase Result

If a complete row contains only the distinguished symbols corresponding to the original attributes, the decomposition is **lossless**.

Here, after applying the FD, the dependency is preserved between the rows, and the decomposition can reconstruct the original relation.

Steps of Chase Algorithm

1. Create the chase table.
2. Create one row for each decomposed relation.
3. Assign common distinguished symbols to common attributes.
4. Apply all functional dependencies.
5. Repeat until no further changes occur.
6. Check whether a complete distinguished row is obtained.
7. If a complete row is obtained, the decomposition is lossless.

Conclusion

The decomposition:

$R(A,B,C) \rightarrow R_1(A,B), R_2(A,C)$

is **lossless** under the given dependency $A \rightarrow B$.

Q7. TEACHER — Multivalued Dependency and 4NF

Given Relation

TEACHER(TID, Subject, Hobby)

Suppose a teacher can teach multiple subjects and can have multiple hobbies.

The subjects and hobbies are **independent** of each other.

Example

TID	Subject	Hobby
T1	DBMS	Cricket
T1	DBMS	Music
T1	Java	Cricket
T1	Java	Music

Teacher T1 teaches two subjects:

- DBMS
- Java

Teacher T1 also has two hobbies:

- Cricket
- Music

Because the subjects and hobbies are independent, all possible combinations are stored.

Multivalued Dependencies

We can represent this using:

TID $\twoheadrightarrow$ **Subject**

and:

TID $\twoheadrightarrow$ **Hobby**

The symbol $\twoheadrightarrow$ represents a **multivalued dependency**.

Problem

The relation creates redundant combinations.

For example:

- T1 + DBMS + Cricket
- T1 + DBMS + Music
- T1 + Java + Cricket
- T1 + Java + Music

If a new hobby is added, multiple rows may need to be inserted.

This causes unnecessary redundancy.

4NF Rule

A relation is in **Fourth Normal Form (4NF)** if, for every non-trivial multivalued dependency:

X $\twoheadrightarrow$ **Y**

X must be a superkey.

Here, TID is not a superkey of the original relation because TID alone does not uniquely identify one row.

Therefore, the relation violates 4NF.

Decomposition

Create two separate relations.

TEACHER_SUBJECT

TEACHER_SUBJECT(TID, Subject)

TID	Subject
T1	DBMS
T1	Java

TEACHER_HOBBY

TEACHER_HOBBY(TID, Hobby)

TID	Hobby
T1	Cricket
T1	Music

Now subjects and hobbies are stored independently.

Conclusion

The decomposition removes the multivalued dependency problem and reduces redundancy.

Therefore, the relations satisfy the requirements of **4NF**.

Q8. Join Dependency and 5NF

Given Relation

SUPPLY(Supplier, Part, Project)

Suppose suppliers supply parts and also work on projects.

There may be independent relationships between:

- Supplier and Part
- Supplier and Project
- Part and Project

Example

Supplier	Part	Project
S1	P1	J1
S1	P1	J2
S1	P2	J1
S1	P2	J2

Here, several combinations are generated.

If these relationships are independent, storing everything in one relation can cause

redundancy.

Decomposition

We can decompose the relation into three smaller relations.

SP Relation

SP(Supplier, Part)

Supplier	Part
S1	P1
S1	P2

*SJ Relation

SJ(Supplier, Project)

Supplier	Project
S1	J1
S1	J2

PJ Relation

PJ(Part, Project)

Part	Project
P1	J1
P1	J2
P2	J1
P2	J2

The original relation can be reconstructed by joining these relations when the appropriate join dependency holds.

Join Dependency

A **join dependency** states that a relation can be reconstructed by joining two or more projections of that relation.

It is represented as:

(R1, R2, R3)

for a decomposition into three relations.

5NF

A relation is in **Fifth Normal Form (5NF)**, also called **Project-Join Normal Form (PJNF)**, when every non-trivial join dependency is implied by the candidate keys.

Advantages

The decomposition:

- Reduces redundancy.
- Prevents unnecessary repeated combinations.
- Makes relationships clearer.
- Helps avoid insertion and deletion problems.
- Allows the original information to be reconstructed through joins.

Conclusion

The relation can be decomposed into:

SP(Supplier, Part)

SJ(Supplier, Project)

PJ(Part, Project)

If the corresponding join dependency is valid, the decomposition achieves **5NF/PJNF**.

Q9. College Database — Anomalies and 3NF

Poorly Designed Relation

Consider:

COLLEGE(StudentID, StudentName, DeptID, DeptName, CourseID, CourseName)

Example Data

StudentID	StudentName	DeptID	DeptName	CourseID	CourseName
101	Ravi	D1	CSE	C1	DBMS
102	Priya	D1	CSE	C2	Java

The department information is repeated for every student belonging to the same department.

This creates redundancy and causes anomalies.

1. Insertion Anomaly

Suppose a new department is created:

D2 – ECE

But currently there are no students in the department.

In the current table, it may be difficult to insert the department information because StudentID and course information are also part of the relation.

Therefore, the database structure causes an **insertion anomaly**.

2. Update Anomaly

Suppose the department name changes from:

CSE → Computer Science and Engineering

If 100 students belong to the CSE department, the department name may appear in 100 rows.

All 100 rows need to be updated.

If one row is not updated, inconsistent data will exist.

This is called an **update anomaly**.

3. Deletion Anomaly

Suppose the last student belonging to a particular department is deleted.

If department information is stored only along with student information, deleting that student may also remove the only record containing information about the department.

This is called a **deletion anomaly**.

Normalization to 3NF

To remove these problems, we divide the large relation into smaller relations.

STUDENT

STUDENT(StudentID, StudentName, DeptID)

StudentID	StudentName	DeptID
101	Ravi	D1
102	Priya	D1

Here:

StudentID → StudentName, DeptID

Candidate key:

StudentID

DEPARTMENT

DEPARTMENT(DeptID, DeptName)

DeptID	DeptName
D1	CSE

Here:

DeptID → DeptName

Candidate key:

DeptID

COURSE

COURSE(CourseID, CourseName)

CourseID	CourseName
C1	DBMS
C2	Java

Here:

CourseID → CourseName

Candidate key:

CourseID

STUDENT_COURSE

If a student can enroll in multiple courses, we create:

STUDENT_COURSE(StudentID, CourseID)

StudentID	CourseID
101	C1
102	C2

The combination:

(StudentID, CourseID)

acts as the candidate key.

Advantages of Decomposition

The normalized structure:

- Removes unnecessary duplication.
- Prevents update anomalies.
- Prevents insertion anomalies.
- Prevents deletion anomalies.
- Improves data consistency.

- Makes database maintenance easier.
- Separates student, department, and course information.

Conclusion

The original COLLEGE relation is decomposed into:

STUDENT

DEPARTMENT

COURSE

STUDENT_COURSE

This removes transitive dependencies and redundancy and provides a properly structured **3NF database design**.

Q10. Attribute Closure and Candidate Keys

Given Relation

R(A, B, C, D)

Functional Dependencies

1. **$A \rightarrow B$**
2. **$B \rightarrow C$**
3. **$C \rightarrow D$**

We need to find the attribute closure and candidate key.

Step 1: Find A^+

Attribute closure means finding all attributes that can be functionally determined by a given set of attributes.

Start with:

$A^+ = \{A\}$

Apply $A \rightarrow B$

Since A determines B:

$A^+ = \{A, B\}$

Apply $B \rightarrow C$

Since B is now available:

$A^+ = \{A, B, C\}$

Apply $C \rightarrow D$

Since C is now available:

$$A^+ = \{A, B, C, D\}$$

No more attributes are available.

Therefore:

$$A^+ = \{A, B, C, D\}$$

Since A^+ contains all attributes of R:

$$A \rightarrow A, B, C, D$$

Therefore, A is a **superkey**.

Because A consists of only one attribute, it is minimal.

Hence:

A is a candidate key.

Checking Other Attributes

B+

Start:

$$B^+ = \{B\}$$

Using $B \rightarrow C$:

$$B^+ = \{B, C\}$$

Using $C \rightarrow D$:

$$B^+ = \{B, C, D\}$$

A is not obtained.

Therefore:

B is not a candidate key.

C+

Start:

$$C^+ = \{C\}$$

Using $C \rightarrow D$:

$$C^+ = \{C, D\}$$

A and B are not obtained.

Therefore:

C is not a candidate key.

D+

There is no functional dependency beginning with D.

Therefore:

D+ = {D}

So D is not a candidate key.

General Method for Finding Candidate Keys

When functional dependencies are given:

Step 1

Write down all attributes in the relation.

Step 2

Choose an attribute or combination of attributes.

Step 3

Calculate its closure.

Step 4

Apply the functional dependencies repeatedly until no new attributes can be added.

Step 5

If the closure contains **all attributes**, the selected set is a superkey.

Step 6

Check minimality. If removing any attribute causes it to stop determining all attributes, it is a candidate key.

Final Answer

For:

R(A,B,C,D)

with:

- $A \rightarrow B$
- $B \rightarrow C$
- $C \rightarrow D$

we get:

A+ = {A,B,C,D}

Therefore:

Candidate Key = {A}

Conclusion: Attribute closure is a useful method for finding candidate keys and determining whether a particular set of attributes can uniquely identify every tuple in a relation.